Intro to Zombie – Person and Zombie Classes

Imagine a scenario where the world has been overrun by zombies. You are stuck on campus in the CBT building with 24 other students. Luckily, a few people happen to have firearms and ammo in their trucks, so you have some protection. Your group also raided the campus kitchens to gather enough rations to survive for a short time. Occasionally, your outer perimeter is overwhelmed by the shambling horde, and you must fight them off or perish. Your only hope is to survive long enough for the National Guard to respond to your signal.

You will spend the next few units writing a program that simulates this zombie apocalypse. This unit focuses on writing **classes for Person and Zombie**, testing them together in a main class, and applying the **Single Responsibility Principle (SRP)**. You should also keep your classes **SIMPLE**, meaning each class should have only **one reason to change** and each method should do **one clear task**.

Requirements

Person Class

The Person class is responsible for **storing and managing the state of a human**. Each instance should:

- Keep track of the person's **food**
- Keep track of the person's **ammo**
- Keep track of the person's **health**
- Keep track of the person's **name**
- Keep track of the person's **status** (boolean isLiving)
- Provide **getters and setters** for all instance variables
- Have a toString() method for easy display

  SRP/SIMPLE note: Person should only handle **state**. It should not decide combat rules or modify zombies.

Zombie Class

The Zombie class is responsible for **storing and managing the state of a zombie**. Each instance should:

- Keep track of **health**
- Keep track of **status** (boolean isLiving)
- Provide **getters, setters, and a toString() method**

  SRP/SIMPLE note: Zombie only stores zombie state. Combat logic should not be inside this class.

Battle Class

The Battle class is responsible for **resolving combat between a Person and a Zombie**. It should include:

Gun Battle

- Method: boolean gunBattle(Person p, Zombie z)
- Returns true if the person kills the zombie, false if the person dies
- Rules:
  - A gun has a **1 in 5 chance** of hitting a zombie
  - If the hit is successful, there is a **1 in 3 chance** of instantly killing the zombie
  - If the hit is not critical, it does **2 damage**
  - It takes **2 non-critical hits** or 1 critical hit to kill a zombie
  - If the hit misses, there is a **1 in 2 chance** the zombie instantly kills the person
  - If the zombie hit is not critical, it does **1 damage**
  - **Each shot uses one round of ammo**

Melee Battle

- Method: boolean meleeBattle(Person p, Zombie z)
- Returns true if the person kills the zombie, false if the person dies
- Rules:
    - Melee attack has a **1 in 3 chance** of hitting a zombie
    - If the hit is successful, there is a **1 in 10 chance** of instantly killing the zombie
    - If the hit is not critical, it does **1 damage**
    - The person **dies** if the attack misses

SRP/SIMPLE note: Battle applies the combat rules. It modifies the states of Person and Zombie, but does not store health or ammo itself. This keeps each class focused on **one responsibility**.

Optional Utility Classes

GameConfig

- Holds **constants** used throughout the program (e.g., starting health, ammo, probabilities)
- Only contains **static constants**
- **Do not put methods or logic here**

RandomUtil

- Handles **random number generation**
- Example methods:
    - public static int randomInRange(int min, int max)
    - public static boolean chance(int outOf)
- Keeps randomness separate from combat logic

SRP/SIMPLE note: Keep GameConfig and RandomUtil separate. Each class has a single reason to change.

Main Class

The main class is responsible for **running the simulation**, not for combat logic or storing person/zombie state.

- Create an array of **names** for the Person objects
  Example: String[] names = {"Jim", "Joe", "Jeff", "Jane", "Jenny", "Julie"};
- Create an array of Person objects using the names array
- Create an array of Zombie objects with a random length (between 15 and 45)
- Include a method int randomInRange(int min, int max) (can be part of RandomUtil)
- **Battle loop:**
    - While there are people and zombies alive:
        - Call gunBattle() if the current person has ammo
        - Call meleeBattle() if the current person has no ammo
    - Loop until either all people or all zombies are dead
- Print a message indicating whether the people survived and how many remain

SRP/SIMPLE note: Main controls the flow of the program only. It does not calculate probabilities, determine damage, or store state for individual characters.

Final Notes

- All **instance variables must be private**
- If a method gets long or code is repeated, create **helper methods**

- Provide **clear comments** explaining what your methods do and why
- You are encouraged to **add extras**:
    - Multiple weapons
    - Armor
    - Skill levels
    - Healing items
    - Stronger zombies

Focus on building **fluency with writing classes**, following **SIMPLE design**, and keeping **each class focused on a single responsibility**.